



SIMATS
ENGINEERING



SIMATS
Sreevidya Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Railway e-Ticket Booking Concurrent Request System

Supervisor:
Dr. Jai sharma

Student name:
R lakshana-1972260027
Thamarai selvan-1965260041
Lakshitha-1924260027



TABLE OF CONTENT



- ☐ Abstract
- ☐ Introduction
- ☐ Problem Identification
- ☐ Analysis
- ☐ Solution Design
- ☐ Implementation
- ☐ Module 1
- ☐ Module 2
- ☐ Module 3
- ☐ Conclusion
- ☐ Future Implementation
- ☐ Reference



ABSTRACT



Physical Booking Offices: Ticket counters were the sole point of access - creating a single bottleneck for all passengers regardless of demand.

Time-Intensive Processes: Inquiry, seat selection, and payment required standing in long queues - often for hours during peak travel seasons.

A Human Error Risk: Manual seat allocation and handwritten records introduced frequent errors in bookings, pricing, and availability.





INTRODUCTION



Automate processes: Eliminates long physical queues and paperwork at railway stations.

Real-time access: Provides live updates on seat inventory, schedules, and train running status.

User convenience: Allows travelers to search, book, modify, or cancel tickets from any device at any time

Search and discovery: Lookup routes, train numbers, timings, and fare structures across different travel classes.

Passenger management: Securely capture traveler details like name, age, gender, and berth preferences.

PROBLEM IDENTIFICATION: the hidden cost

Inefficiency: Manual data entry created significant errors in seat records, fare calculations, and passenger manifests - undermining trust in the system.

Resource Drain: High staffing overhead was required at physical stations, with limited scalability during holiday surges /peak travel windows.

Limited Access: Passengers had no real-time visibility into train delays, seat availability, or fare changes - forcing reliance on outdated information.





ANALYSIS: defining the requirements



User Needs: 24/7 booking availability from any device Real-time seat status and live availability Secure, instant payment processing.

Admin Needs: Centralized database and record management Dynamic train schedule updates System security and access control.

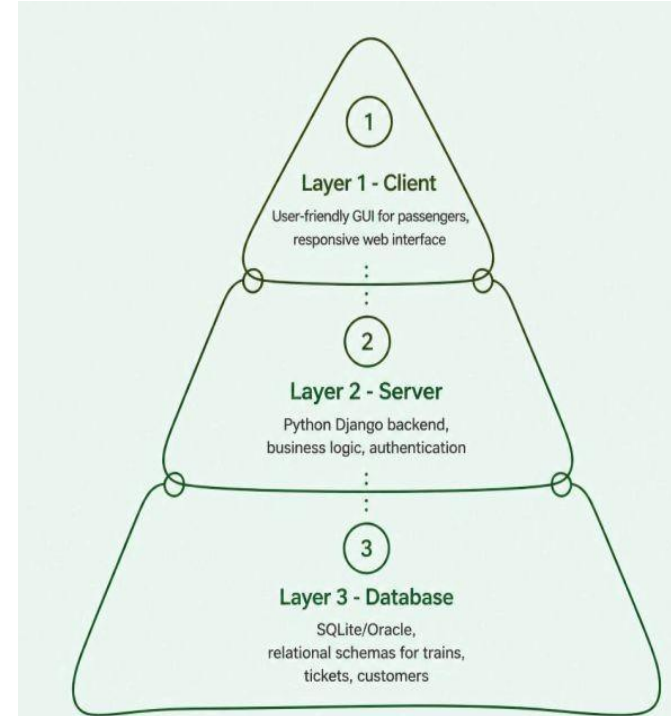
Exploration and Discovery: transforming insights and inspiration into early visual directions through research and experimentation.

SOLUTION DESIGN: architectural foundation

Client-Server Model: A clean separation between the passenger-facing GUI and the backend ensures scalability and maintainability.

Robust Backend: Relational databases like SQLite or Oracle maintain data integrity across millions of concurrent transactions.

Secure Protocols: Authentication layers and encrypted transaction handling protect passenger data and payment information.





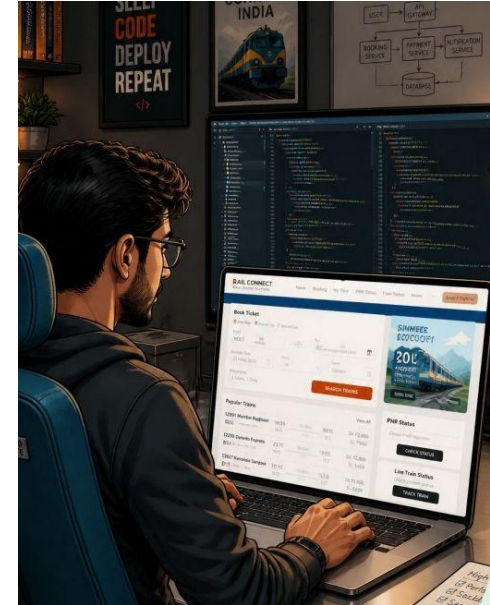
IMPLEMENTATION: the technical stack



Frontend: HTML, CSS, and Bootstrap deliver a responsive, accessible interface across desktop and mobile devices.

Logic Layer: Python Django handles serverside operations, routing, authentication, and seamless data processing.

Database: Structured schemas map customers, trains, tickets, and employees with referential integrity and fast queries.

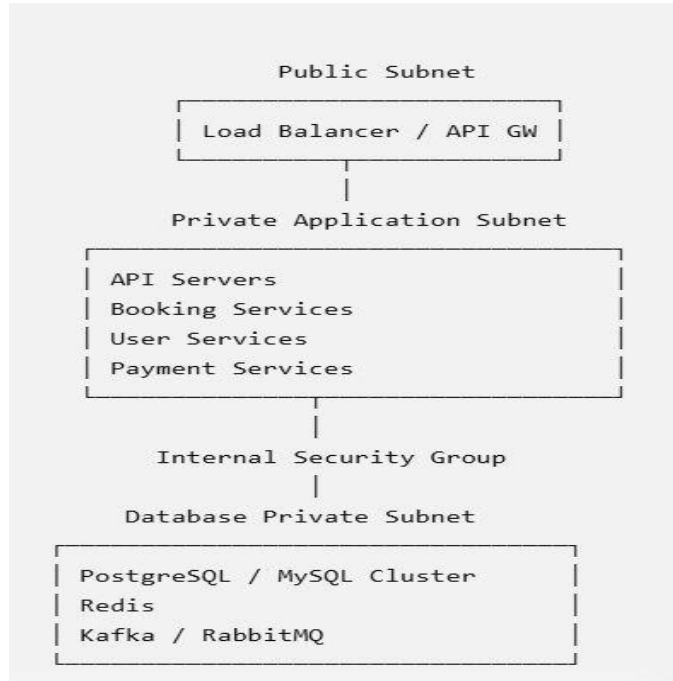




MODULE 1: Booking Server Network Design



A **booking server network design** depends on the type of booking system (hotel, airline, movie tickets, appointments, etc.), but a scalable architecture generally follows the same principles.



Component & Purpose

Load Balancer: Distributes requests across API servers

API Gateway: Authentication, rate limiting, routing

Booking Service: Creates, updates, cancels bookings

Availability Service: Checks inventory (rooms, seats, slots)

Payment Service: Handles payment transactions

Redis: Caches availability and sessions

Database: Stores bookings, users, inventory

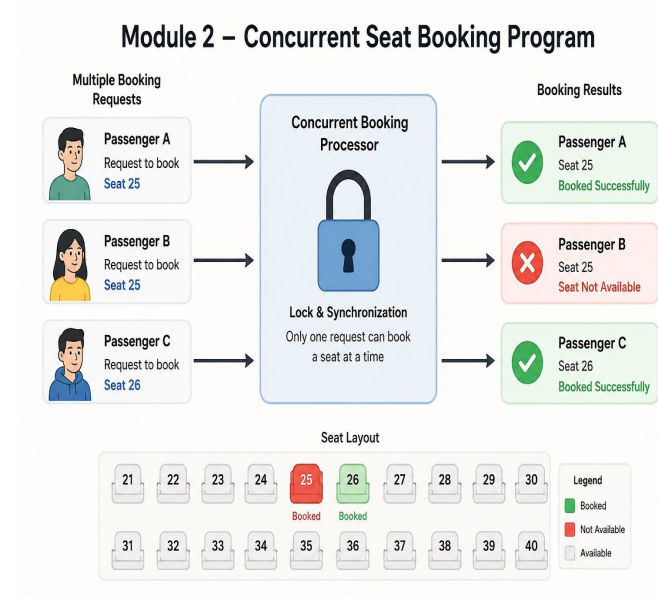
Queue: Processes asynchronous tasks

Notification Service: Sends email/SMS confirmations

MODULE 2- Concurrent Seat Booking Program



- Handles **multiple seat booking requests simultaneously**.
- Checks seat availability before booking.
- Uses **thread synchronization/locks** to prevent double booking.
- Updates the seat status after successful booking.
- Confirms the booking or displays **“Seat Not Available.”**
- Ensures **safe and accurate seat allocation** during concurrent requests.



MODULE 3- Fairness and Response Time verification



- Fairness Verification
 - Prevents users from being unfairly skipped.
 - Ensures correct seat allocation during heavy traffic.
 - Maintains accurate waitlist and RAC assignment.
- Response Time Verification
 - Measures system performance under concurrent bookings.
 - Confirms that users receive timely booking confirmations or failures.
 - Helps identify bottlenecks during peak demand, such as Tatkal booking windows.





Engineer to Excel

SAMPLE CODE



```
trains = {
    1: {"name": "Express 101", "source": "Chennai", "destination": "Bangalore", "seats": 5},
    2: {"name": "Superfast 202", "source": "Delhi", "destination": "Mumbai", "seats": 3},
    3: {"name": "Intercity 303", "source": "Hyderabad", "destination": "Pune", "seats": 4}
}

bookings = []

def display_trains():
    print("\nAvailable Trains")
    print("-" * 60)
    for train_id, train in trains.items():
        print(f"Train ID: {train_id}")
        print(f"Train Name: {train['name']}")
        print(f"Route: {train['source']} -> {train['destination']}")
        print(f"Available Seats: {train['seats']}")
        print("-" * 60)

def book_ticket():
    display_trains()
    try:
        train_id = int(input("Enter Train ID: "))
        if train_id not in trains:
            print("Invalid Train ID!")
            return

        if trains[train_id]["seats"] <= 0:
            print("No seats available!")
            return

        name = input("Enter Passenger Name: ")
        age = input("Enter Age: ")

        trains[train_id]["seats"] -= 1

        bookings.append({
            "name": name,
            "age": age,
            "train_id": train_id,
            "train_name": trains[train_id]["name"]
        })
```



Engineer to Excel



```
        print("\nTicket Booked Successfully!")

    except ValueError:
        print("Invalid Input!")

def view_bookings():
    if not bookings:
        print("\nNo bookings found.")
        return

    print("\nBooked Tickets")
    print("-" * 60)
    for i, ticket in enumerate(bookings, start=1):
        print(f"Booking {i}")
        print(f"Passenger: {ticket['name']}")
        print(f"Age: {ticket['age']}")
        print(f"Train: {ticket['train_name']}")
        print("-" * 60)

def cancel_ticket():
    if not bookings:
        print("\nNo bookings to cancel.")
        return

    view_bookings()
```

```
try:
    choice = int(input("Enter Booking Number to Cancel: "))

    if 1 <= choice <= len(bookings):
        ticket = bookings.pop(choice - 1)
        trains[ticket["train_id"]]["seats"] += 1
        print("Ticket Cancelled Successfully!")
    else:
        print("Invalid Booking Number!")

except ValueError:
    print("Invalid Input!")

def main():
    while True:
        print("\n===== Railway Ticket Booking System =====")
        print("1. View Trains")
        print("2. Book Ticket")
        print("3. View Booked Tickets")
        print("4. Cancel Ticket")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            display_trains()
        elif choice == "2":
            book_ticket()
        elif choice == "3":
            view_bookings()
        elif choice == "4":
            cancel_ticket()
        elif choice == "5":
            print("Thank you for using the Railway Ticket Booking System!")
            break
        else:
            print("Invalid Choice! Please try again.")

if __name__ == "__main__":
    main()
```



APPLICATIONS



 **RAILPULSE**

DashboardMy bookings

Service active

CAPSTONE BOOKING DESK

Move with confidence.

Simple, dependable rail travel between the cities that keep India moving.

Find a train → Live availability



NETWORK STATUS
All routes operational

AVAILABLE TRAINS
03

OPEN SEATS
11

YOUR BOOKINGS
01

Updated just now

01 / SELECT YOUR ROUTE

Available departures

Today: 18 Jun 2025

01

4 seats open

Express 101

Chennai → Bangalore

Departure city Arrival city

06:40 – 11:25 Book now >

02

3 seats open

Superfast 202

Delhi → Mumbai

Departure city Arrival city

06:40 – 11:25 Book now >

03

4 seats open

Intercity 303

Hyderabad → Pune

Departure city Arrival city

06:40 – 11:25 Book now >



Conclusion



The **Railway e-Ticket Booking Concurrent Request System** provides a reliable and efficient solution for managing railway ticket reservations.

The system is designed to handle **multiple booking requests simultaneously** while maintaining accurate seat availability.

Overall, this project demonstrates how **concurrent programming can be applied to real-world railway reservation systems** to improve **performance, reliability, accuracy, and user experience**.



Future Implementation



Online Payment Integration – Add secure payment gateways for ticket transactions.

Real-Time Seat Availability – Display updated seat availability instantly.

Mobile Application – Develop Android/iOS apps for easier booking.

Waitlist Management – Automatically manage cancellations and waitlisted passengers.

AI-Based Demand Prediction – Predict peak booking periods and optimize seat allocation.

Cloud Deployment – Support large numbers of users with scalable cloud infrastructure.



Reference



1. Andrew S. Tanenbaum and Herbert Bos, **Modern Operating Systems**, Pearson.
2. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, **Operating System Concepts**, Wiley.
3. Andrew S. Tanenbaum and Maarten Van Steen, **Distributed Systems: Principles and Paradigms**, Pearson.
4. Python Software Foundation, **Python Documentation – Threading and Synchronization**.



-:THANK YOU:-